

10주차_YOLO&ViT

YOLO

overview

- class+local 단일 문제로 판단
- 실시간 탐지

object detection

- classification: 무엇인지
- localization: 어디에 있는지

object detection 모델의 진화

- DPM
- R-CNN: CNN으로 물체 탐지 진행
 - 느림, endtoend 학습 불가

1stage vs 2stage

- 1stage: class, local 동시에(YOLO)
- 2stage: 순차적으로

YOLO 탄생 아이디어

- 한번에 처리해서 단순화, 속도 높이자!
- 하나의 convolution net 사용

- 예측할 때 전체 이미지를 input으로 사용. 객체의 위치에 대한 특성도 학습

Method

- unified detection: train에서는 사용x, 최종 score 계산

사용한 network

- pascal voc

Training

- pre-training → detection 더 세밀하게 봐야 하기 때문에 해상도 448로 높임
- sse 사용 → local과 class 비율 안맞음. 에러를 동등하게 취급
 - 해결: w와 h의 sqr root 예측

Inference

- YOLO의 추론 과정
- 학습과 동일하게 테스트도 네트워크 1회 실행으로 감지
- 문제: 같은 객체에 대해 중복 박스 생성
 - 해결: NMS
 - NMS의 한계: 근접 객체 제거의 위험이 있음 → Soft-NMS로 개선

Comparison: Others vs YOLO

- Fast
- multibox: 일부 박스만.
- overfeat: YOLO가 더 안정.

- multigrasp

Error analysis

- YOLO는 위치를 정확하게 못 예측하는 경우가 많았음(클래스는 탐지 ㅇㅋ)
- FAST R-CNN은 위치는 잘 맞추는데 배경을 탐지 못하는 에러가 많았음.
- → 앙상블하면 어떨까?
 - 결합했더니 3.2% 향상
 - 단순 앙상블 효과가 아닌 서로의 오류 패턴을 보완한 결과(단일모델론 얻기 힘들)

YOLO의 일반화

- 그림 속 인물 탐지 시험
 - R-CNN: selective search. 자연엔 굿. 아트웍에선 성능 낮음
 - DPM: 비교적 안정적
 - YOLO: 일반화 성능 우수

Conclusion

- 객체 탐지 패러다임을 바꾼 모델
- 최초의 실시간 탐지 모델

QnA

- NMS의 과정이 있는데 look once?
 - 학습할 땐 후처리과정x. endtoend로 학습을 한다에 초점
 - 뽑아낼 땐 한번 보는게 아니지만 학습에선 한번만에 가능이기 때문
- fast r-cnn 에서는 앵커박스등을 쓰는데 여기서 x. v2와 v3에서는 사용. 실시간 학습을 버리고 이걸 넣어서 작은 객체 탐지에 집중하면 조금 느려지지 않을까?

- ancho box 사용이 학습이 더 쉬워서 도입한 듯
-

ViT

Abstract

- Transformer 등장 → NLP 혁신적 변화
- CNN 한계 → Vision에도 transformer 적용 시도

Introduction

- NLP: Transformer → Self-Attention, 효율성·확장성 ↑
- CV: CNN 중심, Attention 결합 시도 but 한계
- Transformer: 스케일↑ → 성능↑, Vision 적용 연구 진행
- Inductive Bias:
 - CNN: 지역성, 이동불변성
 - RNN: 순차성
 - Transformer: bias 약 → 데이터 많이 필요
- 성능:
 - 중간 데이터셋 → ViT < ResNet
 - 대규모 데이터셋 → ViT > CNN, SOTA

Related Work

- Self-Attention vs Conv Layer:
 - 2×2 패치 → Full Self-Attention 적용
 - 대규모 사전학습 시 Transformer도 CNN과 경쟁 가능

- 한계: 저해상도만 처리 → ViT: 중간 해상도까지 확장
- ViT 이전 연구:
 - Bello et al. (2019): CNN 출력 + Self-Attention 보강
 - Chen et al. (2020): 픽셀 단위 sequence로 Transformer 적용, 해상도/공간 축소
- **ViT의 목적**
 1. 대규모 데이터셋에서 CNN 능가 여부 검증
 2. 일반 Transformer를 CV에 직접 적용
 3. Medium-resolution 이미지까지 적용
 - ⇒ NLP에서의 scaling success → CV에도 확장 가능성 제시

Method

- **ViT 구조**
 - 이미지를 패치로 분할 → Flatten → Linear Projection
- **Patch Embedding**
 - 이미지 ($H \times W \times C$) → ($P \times P$) 패치 분할
 - Flatten 후 D차원 벡터로 변환
- **CLS Token & Position Embedding**
 - [CLS] Token → classification 용
 - 위치 정보 부족 → 1D Position Embedding 추가
- **Transformer Encoder**
 - 입력: Patch + [CLS] + Position Embedding
 - 블록: LN → Multi-Head Attention → Residual → LN → MLP → Residual
 - 출력: [CLS] Token → 분류
- **CNN vs ViT**
 - CNN: 강한 inductive bias

- ViT: bias 약함, global 관계 학습 가능, 데이터 충분히 크면 성능↑
- **Hybrid 구조**
 - CNN Feature Map 추출 + ViT 결합 (공간정보 보존, 작은 patch 사용)
- **Fine-tuning & Higher Resolution**
 - Pre-trained Head 제거 → 새 FC Layer 추가
 - 해상도↑ 시 patch 크기 동일 유지 → 긴 시퀀스 생성
 - Position Embedding 불일치 → 2D Interpolation으로 재사용

Method

- **ViT 구조:** 이미지를 패치로 분할 → Linear Projection → Transformer Encoder 입력 → [CLS] 토큰으로 분류
- **Patch Embedding:** 이미지 패치 → Flatten → D차원 벡터 변환
- **CLS Token:** 분류용 토큰 추가, 최종 출력에 사용
- **Position Embedding:** 패치 간 위치 정보 보완 (1D 임베딩)
- **Transformer Encoder:** LN → Multi-Head Attention → Residual → MLP → Residual
- **CNN vs ViT:** CNN은 강한 inductive bias, ViT는 global 관계 학습
- **Hybrid:** CNN feature map + ViT 결합
- **Fine-Tuning:** Pre-trained 모델에 FC 레이어 교체, downstream 학습
- **High Resolution:** 더 큰 해상도 fine-tuning, 2D interpolation으로 position embedding 조정